

3.IMAGE ENHANCEMENT USING INTENSITY TRANSFORMATIONS

AIM:

To study and perform image enhancement using the basic gray-level point transformations.

INTRODUCTION:

The principal objective of image enhancement is to process an image so that the result is more suitable than the original image for a specific application. The word '**specific**' implies that the techniques discussed below are problem-oriented.

Image enhancement can be done in two domains:

1. **Spatial Domain**
2. **Transform Domain**

In **spatial domain image processing**, there are two main approaches:

- **Point processing** – Single pixel is processed at a time
- **Neighborhood processing (Mask processing)** – Convolution of a 3×3, 5×5, or other-sized mask with the image

POINT OPERATIONS:

Point operations are also referred to as **gray-level transformations** or **spatial transformations**. These are among the simplest enhancement techniques. They can be expressed as:

$$G(x,y)=T(F(x,y)) \quad G(x, y) = T(F(x, y)) \quad G(x,y)=T(F(x,y))$$

Where:

- $F(x,y)$ $F(x, y)$ $F(x,y)$ is the original image
- $G(x,y)$ $G(x, y)$ $G(x,y)$ is the transformed image

It can also be expressed as:

$$s=T(r) \quad s = T(r) \quad s=T(r)$$

Where:

- T is the transformation function
- r is the original pixel value
- s is the transformed pixel value

The transformation function T maps pixel value r into pixel value s . These functions are typically implemented using **look-up tables** (LUTs). For an 8-bit grayscale image, the LUT holds values between **0 and 255**.

LINEAR OPERATIONS:

Identity Transformation: The identity function is the trivial case in which output intensities are identical to input intensities.

Negative Transformation: The negative point transformation function (also known as contrast reverse). The negative transformation is used to make the output more suitable for the task at hand (e.g., by making it easier to notice interesting details in the image). The mathematical form for negative point transformation with gray levels in the range $[0, L-1]$ is

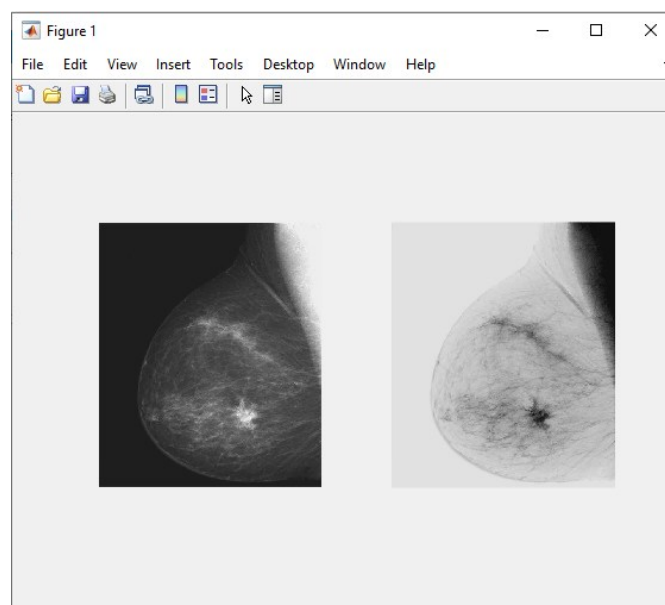
$$s = (L-1) - r$$

NEGATIVE TRANSFORMATION:

Code:

```
Editor - C:\Users\DELL\Documents\MATLAB\untitled.m
untitled.m x untitled4.m x +
1 clear all,close all,clc
2
3 r=imread("C:\Users\DELL\Pictures\IPLAB_Final\test8.tif");
4
5 s=255-r;
6 subplot(1,2,1),imshow(r)
7 subplot(1,2,2),imshow(s)
8
```

Output:



LOGARITHMIC TRANSFORMATIONS:

The log transformation maps a narrow range of low-level grey scale intensities into a wider range of output values and similarly maps the wide range of high-level grey scale intensities into a narrow range of high level output values. The log transformation curve shown in fig. 1, is given by the expression,

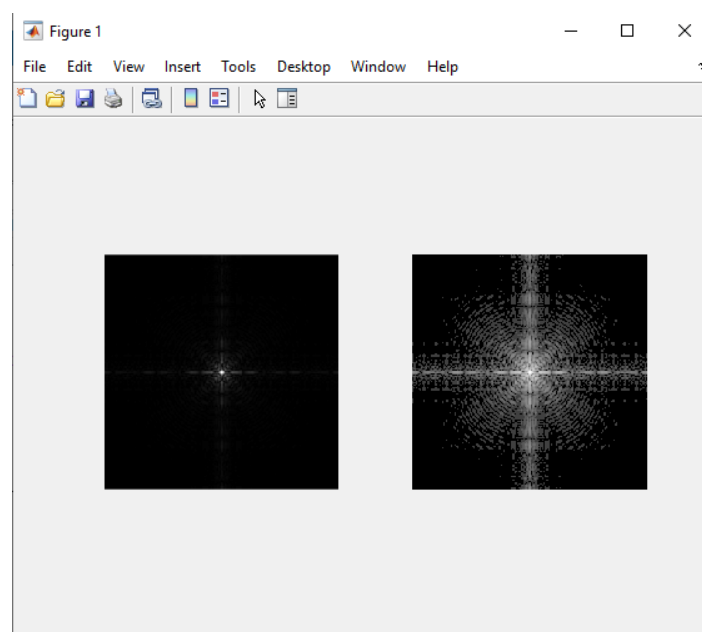
$$s = c \log (1 + r)$$

Where c is a constant. We would use a transformation of this type to expand the values of dark pixels in an image while compressing the higher-level values. The opposite is true of the inverse log transformation.

Code:

```
Editor - C:\Users\DELL\Documents\MATLAB\untitled4.m
untitled.m  untitled4.m  +
1
2  clear all,close all,clc
3
4  I=imread("C:\Users\DELL\Pictures\IPLAB_Final\test11.tif");
5  r=double(I);
6  c=45.9859;
7  s=c.*log(1+r);
8  subplot(1,2,1),imshow(I)
9  subplot(1,2,2),imshow(uint8(s),[])
10 max=max(max(I))
```

Output:



POWER LAW (GAMMA) TRANSFORMATION:

The power law transformation function is described by

$$s = c r^\gamma$$

where r is the original pixel value, s is the resulting pixel value, c is a scaling constant, and γ is a positive value.

As in the case of the log transformation, power-law curves with fractional values of γ map a narrow range of dark input values into a wider range of output values, with the opposite being true for higher values of input levels. Unlike the log function, however, we notice here a family of possible transformation curves obtained simply by varying γ . By convention, the exponent in the power-law equation is referred to as gamma. The process used to correct this power-law response phenomenon is called gamma correction.

Code:

```
clear all, close all, clc
I=imread('C:\Users\hp\OneDrive\ドキュメント\IP IMA
r=double(I);
c=28.3333;
gamma=0.4;
s=uint8(c.*(r.^gamma));
s=c.*log(1+r);
subplot(1,2,1),imshow(I)
subplot(1,2,2),imshow(uint8(s),[])
max=max(max(I))
```

Output:



Code:

```
clear all,close all,clc
I=imread("C:\Users\hp\OneDrive\ドキュメント\IP IMAGES\DIP3E_Original_Images_
r=double(I);
c=28.3333;
s=uint8(c.*(r.^2));
subplot(1,2,1),imshow(I)
subplot(1,2,2),imshow(s)
```

Output:

